UPSC Notes Portal — 21-Day Plan (Spring Boot + React)

Stack: **Spring Boot 3** (REST, JPA, Security, JWT) · **React + Vite** · **PostgreSQL** · **Cloudflare R2** (PDFs) · Pages + Render/Neon deploy
Time: **2–3 hours/day** (≈21 days). Full-time: ~10–12 days. Storage: **10 GB free** on R2, **50 GB** app cap.

> **Learning outcome:** You build a real REST API (not Supabase), secure admin APIs, file upload to S3-compatible storage, and a React SPA for students + admin. Students never log in; PDFs are served from R2 URLs, not streamed through Tomcat.
>
> **No Docker Desktop?** Use **Neon** (free cloud Postgres) or **Homebrew PostgreSQL** locally. For files, use a **local folder** in dev or **Cloudflare R2** directly — MinIO/Docker is optional.

Week 1 — Spring Boot foundation (Days 1–7)

Day	Track	Tasks	Done when
1	Setup	• Install JDK 17+ (OpenJDK), IntelliJ or VS Code, Git — **Docker optional** • Create GitHub repo `Project_sankalpa`; folders `backend/`, `frontend/` • **Database (no Docker):** Neon.tech free Postgres *or* `brew install postgresql@16` + create DB `sankalpa` • Spring Initializr: Web, Validation, JPA, PostgreSQL, Flyway, Lombok (optional) • `application-dev.yml` → JDBC URL; `GET /actuator/health` or `/api/health`	App starts; health returns OK; Postgres connected
2	JPA	• Flyway `V1__subjects.sql` — table + seed (Polity, History, …) • Entity `Subject`, `SubjectRepository` • `SubjectService`, `PublicSubjectController` → `GET /api/public/subjects` • Response DTOs (don't expose entities directly)	Postman/curl lists all subjects as JSON
3	JPA	• Flyway `V2__materials.sql` — subject FK, day_number, topic, title, content_type, object_key, file_size_bytes, published • Entity `Material`, `MaterialRepository` with query methods • `GET /api/public/materials?subjectSlug=&day=&topic=` — only `published=true` • `GET /api/public/materials/{id}`	Filter by subject slug works; unpublished hidden
4	Spring	• Service layer for all public reads; @Transactional where needed • Bean Validation on DTOs (@NotBlank, @Min for day) • `@RestControllerAdvice` — 404, 400 JSON errors • Seed 2–3 test materials via Flyway or `data.sql` (no real PDF yet)	Clean error JSON; layered architecture in place
5	Security	• Add Spring Security; `SecurityFilterChain`: permit `/api/public/**`, `/api/auth/login` • Flyway `V3__admin_user.sql` — admin email + BCrypt hash (from env at first run or fixed seed) • `AdminUser` + `UserDetailsService` (single admin) • CSRF off for stateless API; CORS placeholder for localhost:5173	Public GET works without token; admin routes return 401

Day	Track	Tasks	Done when
6	JWT	• JWT dependency; `AuthController` `POST /api/auth/login` → access token • `JwtAuthenticationFilter` before UsernamePasswordAuthenticationFilter • Secure `/api/admin/**` with `hasRole("ADMIN")` • Postman: login → use Bearer token on admin call	Login returns JWT; admin endpoint succeeds with header
7	Admin API	• `AdminMaterialController`: PATCH publish/unpublish, PUT metadata, DELETE • `GET /api/admin/materials` (includes unpublished) • Unit test 1–2 repository or service tests (JUnit 5)	Full metadata CRUD without file upload yet

Week 2 — Files (Spring) + React student UI (Days 8–14)

Day	Track	Tasks	Done when
8	Storage	• `StorageService` interface: putObject, deleteObject, publicUrl • **No Docker:** impl `LocalFileStorageService` → save under `./data/uploads`; expose URL via Spring resource handler or copy path to DTO • **Or:** skip to R2 on Day 11 · **Or with Docker:** MinIO via Colima/compose • Config via `@ConfigurationProperties` (`storage.type=local|r2|minio`)	File saved; URL/path works in API response
9	Upload	• `POST /api/admin/materials` multipart: file + JSON (subjectId, day, topic, title, type) • Stream to MinIO; save Material with objectKey, fileSizeBytes, published=true • Return DTO with `fileUrl` for React	Real PDF upload creates DB row + object in MinIO
10	Quota	• `StorageQuotaService` — sum file sizes; reject if > 50 GB • `GET /api/admin/storage-stats` → usedBytes, capBytes, warn at 45 GB • Public material DTO includes fileUrl pointing to MinIO/R2, not Spring download	Upload blocked when quota exceeded (test with low cap in dev)
11	R2	• Cloudflare R2 bucket; API token; `application-prod.yml` S3Client → R2 endpoint • Same `StorageService` impl with profile switch MinIO vs R2 • Upload one PDF to R2; verify public URL in Material response	Prod storage config documented in docs/SETUP.md
12	React	• `npm create vite@latest frontend -- --template react-ts` • React Router, axios, Tailwind (or CSS modules matching mockup) • `.env` `VITE_API_BASE_URL=http://localhost:8080` • Layout: header “Sankalpa UPSC”, mobile-first	Dev server runs; can call backend health/subjects
13	React	• Home page: subject tile grid from `GET /api/public/subjects` • Route `/subjects/:slug` — material cards from public materials API • Filters: day dropdown, content type; client or query params	Matches student mockup (tiles + list)
14	React	• Route `/materials/:id/view` — embed PDF.js with fileUrl from API • Download button → direct R2/MinIO URL • Loading + error states; empty subject message	Student preview/download without admin UI

Week 3 — React admin, deploy, launch (Days 15–21)

Day	Track	Tasks	Done when
15	React Admin	• Routes `/admin/login`, `/admin/*` protected (no token → redirect) • Login form → POST /api/auth/login; store JWT in sessionStorage • Axios interceptor adds Authorization header	Admin shell + sidebar like mockup
16	React Admin	• Upload page: form fields + file input; FormData multipart to admin API • Progress UI; success toast; link to view on student site	End-to-end upload from browser (admin) to storage
17	React Admin	• Materials table: list, edit metadata, unpublish, delete • Dashboard cards: storage-stats API, count published today	Admin mockup features covered
18	Integration	• CORS: allow Cloudflare Pages origin + localhost • Cache-Control on public GET controllers (e.g. 5 min) • Spring profile prod; env vars for DB, JWT secret, R2 keys (never in git)	Frontend + backend work together locally on phone via LAN IP (optional)
19	Deploy	• Neon (or Railway) PostgreSQL; run Flyway on prod DB • Deploy Spring Boot to Render/Fly.io; set env secrets • Smoke test public APIs on prod URL	Backend live HTTPS
20	Deploy	• `npm run build`; Cloudflare Pages → `frontend/dist` • `VITE_API_BASE_URL` = prod Spring URL • Test admin login + upload on prod; student view PDF from R2	Full stack prod; share pages.dev link
21	Launch	• Upload first real batch (Polity, Current Affairs); compress large scans • Pin WhatsApp/Telegram: one URL, how to use tiles • Checklist: Android + iPhone, mobile data, 401 on admin without login • Optional stretch: presigned PUT upload (Phase 2 learning)	1000-student ready process in place

After Day 21 (optional Phase 2 learning)

- Presigned R2 upload from React (Spring only signs URL — advanced memory/scaling pattern)
- Spring Integration tests with @WebMvcTest + Testcontainers Postgres
- Batch CSV import endpoint; PDF compression script before upload
- PWA manifest; “New today” API filter

Daily habits (post-launch)

When	Action	Time
After class	Admin upload → Publish → one chat message with subject/day link	5–10 min
Weekly	Check storage-stats; unpublish outdated PDFs	15 min

Key Spring topics touched (study map)

Topic	Days
Boot, config profiles, actuator	1
JPA, Flyway, repositories	2–4
REST, DTOs, validation, advice	3–4
Spring Security, JWT	5–6
Multipart, S3 SDK, quotas	8–11
CORS, caching headers, deploy	18–20